

AltTasker - Full Use Case Specifications

Source: AltTasker_Scope_Internal_revised (Section 4 + F1–F12 detail)

Purpose: Ground truth for use case diagram drawing - every `<<include>>` and `<<extend>>` grounded in the feature specs.

Convention used throughout:

- `<<include>>` = the base UC *always* calls this sub-behaviour; it is mandatory and reusable
 - `<<extend>>` = an optional or conditional branch that *extends* the base UC at a named extension point
 - **Preconditions** = what must be true before the UC can start
 - **Postconditions** = what is guaranteed true after successful completion
 - Guard notes after `[IF ...]` describe the condition that activates an extension
-

Notation Key for the Diagram Drafter

UC_BASE	---<<include>>---	UC_SUB	The arrow points FROM base TO the included UC
UC_EXT	---<<extend>>---	UC_BASE	The arrow points FROM the extending UC TO the base

When multiple UCs share the same included behaviour (e.g. `<<include>> Verify Wallet Balance`), draw that sub-UC once and fan the `<<include>>` arrows from all callers.

PART A - CLIENT / CEO FLOWS

UC01 - Submit Project via AI Elicitation Engine

Primary Actor: CLIENT / CEO

Secondary Actor: System (LLM - FastAPI AI Service)

Feature reference: F2

Subscription gate: Client Pro required (HTTP 403 returned before Stage 1 if `sub_client_tier = 'free'`)

Preconditions:

1. Actor is authenticated; `active_role = CLIENT`, `client_subtype = CEO`
2. `users.subscription_client_tier = 'pro'` AND `sub_client_expires_at > now()`
3. No active unfinished elicitation session for this user (system resumes if interrupted)

Main Success Scenario:

1. Actor opens "Post a Project" - system displays behavioral intake prompt (not a form)
2. Actor types free-form symptom description (what hurts; business goal)
3. System runs LLM extraction: intent separation, scale signal extraction, void detection
4. System presents 2–3 archetype options in plain business language
5. Actor selects matching archetype; system locks archetype
6. System runs SDLC injection loop: for each detected void (e.g. ground truth absent), presents mandatory milestone injection with plain-language rationale; actor accepts
7. System presents 4 behavioral architecture probe questions
8. Actor answers all 4 probes
9. System evaluates probe answers against infrastructure thresholds
10. [IF threshold crossed → Stage 4 required] System generates signed tech handoff link
(`client_subtype: TECH_TEAM` in token, 72-hour expiry); actor is hard-blocked from Stage 4
11. Actor sends link to their technical lead
12. TECH_TEAM completes Stage 4 via `UC01t`
13. Synthesis engine resolves CEO vs TECH_TEAM signal conflicts (private reconciliation); produces CapabilityFootprint, Artifact A draft, Artifact B structure
14. Automated quality gate runs: completeness score ≥ 0.70 + matching pre-check + void resolution check
15. [IF all pass] Artifact A auto-published; matching engine generates shortlist; actor notified → `UC04` unlocked
16. ElicitationSession marked `COMPLETED`; Project record created with `spec_state = PUBLISHED`

Extensions:

- `UC01a` [EXTEND at Step 9 - IF infrastructure threshold crossed AND CEO indicates no TECH_TEAM exists]
Scenario A - Technical Discovery Pathway
- `UC01b` [EXTEND at Step 9 - IF actor set `project.self_technical = true`]
Scenario B - Self-Technical CEO completes Stage 4 directly
- [AT Step 14 - IF any quality check fails] System sets `spec_state = RETURNED_TO_CLIENT`; LLM generates targeted advisory note; actor re-enters elicitation at the specific void that failed (not from Step 1). No admin action.

Includes:

- `<<include>> UC03` - Purchase Client Pro Subscription (guard: called as redirect if sub is `free` before Step 1)
- `<<include>> Verify Subscription Gate` - checked at route entry before displaying intake form
- `<<include>> Run SDLC Void Detection` - Step 3 LLM sub-process
- `<<include>> Run Automated Quality Gate` - Step 14 sub-process (shared with re-entry path)

Postconditions (success):

- `projects` row created; `capability_footprints` row created; `artifact_a` row created with `state = PUBLISHED`; `artifact_b` row created with `access_state = LOCKED`
- `match_shortlist` generated (3–5 candidates)
- CEO notified; shortlist visible in dashboard

UC01a - Proceed via Technical Discovery Pathway (Scenario A)

Primary Actor: CLIENT / CEO

Feature reference: F2 Scenario A, F3 TECH_DISCOVERY

Extends: UC01 at extension point - Stage 4 unreachable, no TECH_TEAM available

Preconditions:

1. UC01 Stage 3 probe answers indicate integration with existing system (Stage 4 required)
2. Actor confirms they have no technical team member to send the handoff link to

Main Success Scenario:

1. System presents two options: (a) inject a Technical Discovery milestone as Milestone 0 into the project, or (b) purchase a TECH_DISCOVERY service engagement first
2. Actor selects option (a): system injects **TECH_DISCOVERY** milestone as Milestone 0; elicitation resumes with reduced Artifact A (no technical blueprint sections)
3. Project published; expert bids will include architecture discovery in Milestone 0 scope
4. OR - Actor selects option (b): actor is redirected to **UC11** to purchase a TECH_DISCOVERY service; after that engagement closes, actor may complete Stage 4 manually using the architecture document delivered by the expert

Extends: UC01 (conditional - Stage 4 unreachable + no TECH_TEAM)

Postconditions:

- If option (a): project published with **engagement_type = TECH_DISCOVERY**; Milestone 0 injected
 - If option (b): service engagement created; main elicitation session paused
-

UC01b - Complete Stage 4 as Self-Technical CEO (Scenario B)

Primary Actor: CLIENT / CEO

Feature reference: F2 Scenario B

Extends: UC01 at extension point - **project.self_technical = true**

Preconditions:

1. Actor has set **self_technical = true** during Stage 1 intake
2. Actor is authenticated as **CLIENT / CEO**

Main Success Scenario:

1. System presents Stage 4 form directly to the actor (same form that TECH_TEAM normally sees)
2. Actor inputs: stack tags, integration method, legacy data volume, deployment expectation; optionally uploads sensitive schemas
3. System records inputs; grants actor TECH_TEAM capabilities for this project (**self_technical_projects** JWT claim updated)
4. Synthesis runs; Surface D conflict gate is skipped for all bids on this project

Extends: UC01 (conditional - **self_technical = true**)

Postconditions:

- `project.self_technical = true` persisted
 - Actor's JWT updated with `self_technical_projects: [project_id]`
 - Artifact B populated with actor's Stage 4 inputs; access state = LOCKED until engagement CONNECTED
-

UC01t - Complete Tech Team Architecture Handoff (TECH_TEAM Route)

Primary Actor: CLIENT / TECH_TEAM

Feature reference: F2 Stage 4

Extends: UC01 (TECH_TEAM drives Stage 4 independently on a separate account)

Preconditions:

1. Actor has opened a signed handoff link (`client_subtype: TECH_TEAM` encoded in token)
2. Token is not expired (72-hour window)
3. Actor has registered or logged in; `active_role = CLIENT`, `client_subtype = TECH_TEAM`, `linked_project_id` set

Main Success Scenario:

1. Actor sees security framing: "Your CEO is scoping an AI integration. To protect your system's IP and ensure the hired expert understands your architecture before being selected, please confirm the following."
2. Actor inputs: stack tags (multi-select), integration method, legacy data volume range, deployment expectation
3. Actor optionally uploads sensitive schemas, payload samples, integration contracts - these go into Artifact B vault
4. Actor submits; system writes inputs to Artifact B staging, writes stack tags to Artifact A draft
5. System triggers synthesis engine (UC01 Step 13 onwards)

Guard: Route `POST /projects/{id}/stage4` is guarded by `client_subtype = TECH_TEAM || (CLIENT/CEO && self_technical = true)`. CEO without self_technical flag receives HTTP 403.

Postconditions:

- Artifact B populated with sensitive uploads
 - Synthesis engine triggered; CapabilityFootprint generated
 - TECH_TEAM account linked to `project_id`
-

UC02 - Top Up Platform Wallet via WALLET_TOPUP VA VietQR

Primary Actor: CLIENT / CEO or EXPERT (any wallet-holding user)

Feature reference: F1.5, F10

Subscription gate: None - Free tier can top up

Preconditions:

1. Actor is authenticated; `active_role = CLIENT/CEO` or `EXPERT`
2. Actor has a `wallets` row (created on registration)
3. Actor has a permanent `WALLET_TOPUP` virtual account (`entity_type = WALLET_TOPUP`)

Main Success Scenario:

1. Actor opens "Top Up Wallet" in dashboard
2. System displays VietQR QR code for actor's permanent `WALLET_TOPUP` VA number (no `fixed_amount` - any amount accepted)
3. Actor scans QR with banking app; enters any amount; confirms bank transfer
4. SePay IPN fires: `POST /webhooks/sepay/ipn` (HMAC-verified)
5. NestJS IPN handler: matches `va_number` → `entity_type = WALLET_TOPUP` branch
6. Atomic DB write: `wallet.available_balance += transferAmount`; `wallet_transactions` row written `{ type: TOP_UP }`
7. Actor notified: "Wallet topped up: +{amount} VND. Available balance: {new_balance} VND"

Extensions:

- [IF SePay IPN is a duplicate reference] Idempotency check fires; transaction ignored; no double-credit

Postconditions:

- `wallet.available_balance` increased by transferred amount
- `wallet_transactions` row persisted (immutable ledger)
- Actor sees updated balance in dashboard

UC03 - Purchase Client Pro Subscription from Wallet Balance

Primary Actor: CLIENT / CEO

Feature reference: F1.5

Subscription gate: None for the purchase itself; this UC activates the gate

Preconditions:

1. `active_role = CLIENT`, `client_subtype = CEO`
2. `subscription_client_tier = 'free'` OR subscription has expired
3. `wallet.available_balance >= 500,000 VND` (Client Pro price)

Main Success Scenario:

1. Actor opens "Subscription" panel; sees Client Pro plan (500K VND / 6 months); current tier displayed
2. Actor clicks "Activate Client Pro"
3. NestJS guard confirms `available_balance >= 500,000 VND`
4. Atomic DB transaction: `available_balance -= 500,000`; `wallet_transactions` row `{ type: SUBSCRIPTION }`; `users.subscription_client_tier = 'pro'`; `sub_client_expires_at = now() + 6 months`; `user_subscriptions` audit row written

5. Actor notified: "Your Pro subscription is active until {date}"
6. All LLM-gated routes (elicitation engine, matching engine, Artifact B access, Add-On Phase) now return HTTP 200 for this actor

Extensions:

- [IF `available_balance < 500,000 VND`] System returns 422 { `code: "INSUFFICIENT_BALANCE", top_up_url: "/wallet/topup"` }; actor redirected to UC02
- [AT Step 6 - IF subscription was previously expired with active engagement] Grandfathered engagement continues; new gated features re-enabled

Includes:

- `<<include>>` UC02 - called as prerequisite redirect if balance is insufficient

Postconditions:

- `users.subscription_client_tier = 'pro'`; `sub_client_expires_at` set
- `user_subscriptions` audit row written
- UC01, UC04, UC08, UC10 now accessible

UC04 - View Artifact A and Expert Shortlist

Primary Actor: CLIENT / CEO (primary); CLIENT / TECH_TEAM (read access, shared)

Feature reference: F5, F4

Subscription gate: Client Pro required for full seam gap maps; Free tier sees domain-only match cards

Preconditions:

1. Actor authenticated as `CLIENT / CEO` or `CLIENT / TECH_TEAM`
2. `project.spec_state = PUBLISHED`
3. `match_shortlist` record exists for this project

Main Success Scenario:

1. Actor opens Project dashboard; sees Artifact A (business intent, capability footprint, system physics, milestone framework, Artifact B escrow notice)
2. Actor views expert shortlist (3–5 match cards): each card shows match strength label (Strong / Qualified / Conditional), verified capability summary, seam coverage grid (color-coded), known gaps named explicitly, recommended next step
3. Score numbers are internal; actor sees only the three-tier strength label
4. Actor can click into any match card for full profile view

Extensions:

- [EXTEND: UC05 - CEO on Tier 1 project] Actor can respond to expert spec clarification requests from this view
- [IF `subscription_client_tier = 'free'`] Seam gap map replaced with domain-only view; actor prompted to upgrade for full seam detail

Postconditions:

- No state change - read-only view
 - Actor informed of shortlisted experts; ready to initiate UC07
-

UC05 - Respond to Expert Spec Clarification Request (Surface A - Tier 1 Projects)

Primary Actor: CLIENT / CEO (Tier 1 projects only); CLIENT / TECH_TEAM (Tier 2+ projects - see UC05t)

Feature reference: F6 Surface A

Preconditions:

1. `spec_clarifications.status` = OPEN for a clarification on this project's spec
2. For CEO: project is Tier 1 AND `active_role` = CLIENT/CEO
3. Clarification targets one of: "footprint" | "milestone_framework" | "acceptance_criteria" | "system_physics"

Main Success Scenario:

1. Actor receives notification: "An expert has filed a spec clarification on [component]"
2. Actor opens clarification thread; reads expert's question
3. Actor writes response text; submits
4. NestJS writes response; `spec_clarifications.status` → ANSWERED; response versioned against spec (immutable audit trail)
5. Expert notified; expert can mark clarification RESOLVED (see UC19)

Extensions:

- [IF expert marks RESOLVED] Status → RESOLVED; clarification thread closed

Postconditions:

- `spec_clarifications.status` = ANSWERED
 - Response versioned; expert can proceed with bid
-

UC06b - Approve Expert Selection

Primary Actor: CLIENT / CEO

Feature reference: F6 bid state machine

Preconditions:

1. Bid state is CEO_REVIEW (reached after TECH_APPROVED or CONFLICT_RESOLVED_CEO_OVERRIDE)
2. `active_role` = CLIENT/CEO
3. No open Surface C negotiation in progress

Main Success Scenario:

1. Actor reviews bid in CEO dashboard: match card, architectural approach summary, milestone pricing
2. Actor clicks "Select This Expert"
3. NestJS: `bid.state` → `SELECTED`; all other bids on this project → `DECLINED`
4. Connection request sent to expert (see `UC07`)

Extensions:

- [EXTEND: `UC06d`] Actor can open price negotiation window (Surface C) before approving
- [IF bid was `TECH_DISAPPROVED` and actor wants to proceed] `UC06c` must be completed first; this UC is guarded until `bid.state = CONFLICT_RESOLVED_CEO_OVERRIDE`

Postconditions:

- `bid.state = SELECTED`
 - All competing bids → `DECLINED`
 - `UC07` triggered
-

UC06c - Override TECH_TEAM Not Recommended Flag (Surface D)

Primary Actor: CLIENT / CEO

Feature reference: F6 Surface D

Preconditions:

1. `bid.state = TECH_DISAPPROVED`
2. `active_role = CLIENT/CEO`
3. CEO has reviewed TECH_TEAM's disapproval reason

Main Success Scenario:

1. Actor sees "TECH_TEAM has flagged this bid as Not Recommended" with TECH_TEAM's required reason text
2. Actor chooses to proceed; system presents Surface D override form
3. Actor enters `override_reason` (minimum 50 characters) and `acknowledged_risks` text
4. Submits; NestJS: `bid.state` → `CONFLICT_RESOLVED_CEO_OVERRIDE`; `bid_conflict_overrides` row written; `CEO_REVIEW` unlocked
5. PlatformDecision record written (audit log for Platform Integrity Monitor)

Postconditions:

- `bid.state = CONFLICT_RESOLVED_CEO_OVERRIDE`
 - `bid_conflict_overrides` row persisted (immutable; visible in admin Platform Integrity Monitor)
 - `PlatformDecision` row written
 - `CEO_REVIEW` unlocked; `UC06b` now accessible
-

UC06d - Propose Price Adjustment - Surface C (Max 2 Rounds)

Primary Actor: CLIENT / CEO (proposer); EXPERT (responder - see UC20c)

Feature reference: F6 Surface C

Preconditions:

1. `bid.state` = `TECH_APPROVED` or `CONFLICT_RESOLVED_CEO_OVERRIDE`
2. `active_role` = `CLIENT/CEO`
3. Round counter < 2 (server enforces; returns 422 if `round_number` > 2)

Main Success Scenario:

1. Actor opens price negotiation panel; sees current milestone pricing from bid
2. Actor proposes adjusted amounts per milestone + a message
3. NestJS: `price_negotiations` row written; `bid.state` → `PRICE_NEGOTIATION`; expert notified
4. Expert responds via `UC20c` (ACCEPT / COUNTER / DECLINE)
5. [IF ACCEPT] `bid.state` → `NEGOTIATION_COMPLETE`; actor proceeds to `UC06b`
6. [IF COUNTER on round 1] Actor can accept or counter (round 2)
7. [IF COUNTER on round 2] Actor **must** ACCEPT or DECLINE - no further counter possible; NestJS returns 422 on attempt

Extensions:

- [IF expert DECLINES at any round] Negotiation ends; bid returns to `CEO_REVIEW` at original price; actor can proceed to `UC06b` with original pricing

Includes:

- `<<include>> UC20c` (EXPERT side of the negotiation)

Postconditions:

- `price_negotiations` rows written (1-2)
- Final agreed pricing locked at bid acceptance
- `bid.state` = `NEGOTIATION_COMPLETE` → ready for `UC06b`

UC07 - Send Connection Request to Expert; Complete NDA Click-Through

Primary Actor: CLIENT / CEO

Feature reference: F5 (connection flow)

Preconditions:

1. `bid.state` = `SELECTED`
2. `engagement.state` = `PENDING` (created when bid selected)
3. Expert has not yet accepted or declined

Main Success Scenario:

1. System auto-sends connection request to expert on bid selection (no separate actor action needed)

2. Expert receives notification; reviews Artifact A; accepts via UC21
3. On expert acceptance: `engagement.state` → `CONNECTED`
4. NDA click-through presented to CEO: checkbox acknowledgment
5. CEO completes NDA click-through; `engagement.nda_accepted` = `true`
6. Artifact B becomes queryable by EXPERT and TECH_TEAM (`access_state` = `UNLOCKED`)
7. Messaging thread created (UC31 becomes active for this engagement)

Extensions:

- [IF expert declines] `engagement.state` → `DECLINED`; CEO can select another expert from shortlist (if any remain) or re-open bidding

Postconditions:

- `engagement.state` = `CONNECTED`
- `engagement.nda_accepted` = `true`
- Artifact B `access_state` = `UNLOCKED`
- `messaging_threads` row created

UC08 - Fund Milestone via Per-Milestone VA QR

Primary Actor: CLIENT / CEO

Feature reference: F10 (MILESTONE IPN branch)

Preconditions:

1. `engagement.state` = `CONNECTED` or `ACTIVE`
2. `milestone.state` = `DEFINED` (acceptance criteria and sprint plan set)
3. `active_role` = `CLIENT/CEO`
4. CEO has clicked "Fund Milestone" - per-milestone VA has been generated (`milestone.state` → `AWAITING_PAYMENT`)

Main Success Scenario:

1. Actor clicks "Fund Milestone {n}" in milestone dashboard
2. NestJS: creates per-milestone VA (`entity_type`: `MILESTONE`, `fixed_amount`: `milestone.payment_amount_vnd`, `expires_at`: `now + 24h`); milestone state → `AWAITING_PAYMENT`
3. System displays VietQR QR for exact milestone amount
4. Actor scans QR; transfers exact amount (bank rejects wrong amounts due to fixed VA)
5. SePay IPN fires; NestJS IPN handler matches VA → `entity_type` = `MILESTONE`
6. Atomic DB transaction: `wallet.available_balance -= amount`; `wallet.locked_balance += amount`; `wallet_transactions { type: ESCROW_LOCK }`; `escrow_accounts { status: HELD }`; `milestone.state` → `FUNDED` → `IN_PROGRESS`
7. If pay-gated documents staged for this milestone: `release_state` → `RELEASED` (TECH_TEAM document inbox updated)
8. Both parties notified via Socket.io

Extensions:

- [IF VA expired (24h)] VA status = EXPIRED; actor must re-click "Fund" to generate a new VA
- [IF actor transfers wrong amount] SePay bank rejects transfer; no IPN fires; actor must retry with exact amount

Postconditions:

- `milestone.state = IN_PROGRESS`
- `escrow_accounts.status = HELD`
- `wallet.locked_balance` increased; `available_balance` decreased
- Expert can now begin work; sprint plan creation unlocked (UC23)

UC09 - Approve Business / Budget Milestones - Triggers Escrow Release + Chi Hộ

Primary Actor: CLIENT / CEO (for `sign_off_authority = CEO` or `JOINT`)

Feature reference: F7 Layer 1, F10

Preconditions:

1. `milestone.state = SUBMITTED`
2. `milestone.sign_off_authority = CEO` or `JOINT`
3. For `JOINT`: `TECH_TEAM` has already signed off all technical criteria (`verified_at` set on `TECH_TEAM` criteria)
4. `active_role = CLIENT/CEO`

Main Success Scenario:

1. Actor is notified: "Expert has submitted Milestone {n} for review"
2. Actor opens milestone review panel; reads deliverable statement; reviews acceptance criteria
3. Actor verifies each required acceptance criterion: marks `verified_at` + role
4. Final required criterion verified → NestJS guard checks all `is_required = true` criteria have `verified_at` set
5. `milestone.state → APPROVED`
6. Atomic ledger: `ESCROW_RELEASE - client.locked_balance -= amount;`
`escrow_accounts.status → RELEASED;` `expert.available_balance += (amount - fee);`
`PLATFORM_FEE` row written
7. Chi hộ API called (async): `POST /chiho/v1/disburse { amount, bank_account_xid, reference: "MS-{id}" }`
8. `withdrawal_requests` row written `{ type: MILESTONE_RELEASE, status: PENDING }`
9. On SePay credit IPN confirmation: `withdrawal_requests.status → COMPLETED;`
`milestone.state → RELEASED`

Extensions:

- [IF a required criterion is not met] Actor can request revision: `milestone.state → IN_REVISION;` revision counter increments; expert notified

- [IF actor files a dispute] UC dispute flow; `escrow_accounts.status` → FROZEN

Postconditions:

- `milestone.state` = RELEASED
 - Expert's `available_balance` credited (net of 5% platform fee)
 - Chi hộ transfer initiated; bank confirmation pending
-

UC10 - Approve Add-On Phase - Budget & Timeline Dimension

Primary Actor: CLIENT / CEO

Feature reference: F8

Preconditions:

1. `addon_phase_requests.state` = PENDING_CEO_APPROVAL
2. TECH_TEAM has already approved the technical justification dimension (UC10t completed)
3. `active_role` = CLIENT/CEO

Main Success Scenario:

1. Actor receives notification: "An add-on phase has been proposed. Tech team has reviewed and approved the technical justification. Your review: budget and timeline."
2. Actor opens Add-On Phase Brief: reads causal chain, new capability requirement, why invisible at intake, tech team's approval note
3. Actor reviews budget implication and timeline extension
4. Actor approves
5. NestJS: `addon_phase_requests.state` → APPROVED; new Milestone row automatically generated and appended to engagement; admin receives informational log entry (no approval gate)
6. New milestone displayed in CEO dashboard; actor proceeds to UC08 to fund it

Extensions:

- [IF actor declines] `addon_phase_requests.state` → REJECTED; engagement continues without new milestone; expert notified; messaging thread used to negotiate alternative

Postconditions:

- New `milestones` row created and linked to engagement
 - Actor notified to fund the new milestone
 - Scope evolution log updated (causal chain + outcome - feeds reputation signal at close)
-

UC11 - Buy Expert Service Directly (SERVICE_PURCHASE or TECH_DISCOVERY)

Primary Actor: CLIENT / CEO

Feature reference: F3 Path B, F10 SERVICE IPN branch

Subscription gate: Free tier accessible

Preconditions:

1. `active_role = CLIENT/CEO`
2. `wallet.available_balance >= service_listing.price_vnd`
3. Service listing `state = ACTIVE`

Main Success Scenario:

1. Actor browses Path B marketplace (UC30); selects a service listing
2. Actor clicks "Buy Service"
3. NestJS creates per-order VA (`entity_type: SERVICE, fixed_amount: service.price_vnd, expires_at: now + 24h`)
4. System displays VietQR QR for exact service amount
5. Actor scans and transfers exact amount
6. SePay IPN fires (SERVICE branch): atomic ledger `ESCROW_LOCK; service_engagements { state: FUNDED }` created
7. Expert notified: "New service order received"
8. Service engagement proceeds; expert delivers; CEO approves completion

Extensions:

- [IF `TECH_DISCOVERY` type] After engagement CLOSED: CEO receives architecture document; CEO can optionally complete elicitation Stage 4 using this document (resumes UC01 from Stage 4)

Includes:

- `<<include>> UC30` - actor must browse marketplace first
- `<<include>> UC02` - if wallet balance insufficient, actor redirected to top up

Postconditions:

- `service_engagements.state = FUNDED`
- Expert notified; `EscrowAccount` holding funds

UC12a - Complete Post-Engagement Review (CEO Form)

Primary Actor: CLIENT / CEO

Feature reference: F11

Preconditions:

1. `engagement.state = CLOSED` (all milestones in RELEASED state)
2. CEO has not yet submitted a review for this engagement
3. `active_role = CLIENT/CEO`

Main Success Scenario:

1. Actor receives notification to leave a review
2. Actor opens review form: Overall rating (1–5 stars), Communication clarity rating, Milestone structure effectiveness rating, Open text comment

3. Actor submits; `reviews` row written { `reviewer_role: CEO` }
4. Expert's reputation display updated (average rating, completion count)

Postconditions:

- `reviews` row written; expert reputation updated
- CEO review data feeds platform analytics (F12 Module 6)

PART B - CLIENT / TECH_TEAM FLOWS

UC05t - Respond to Expert Spec Clarification (Surface A - Tier 2+ Projects)

Primary Actor: CLIENT / TECH_TEAM

Feature reference: F6 Surface A

Extends: UC05 (TECH_TEAM variant for Tier 2+ projects)

Preconditions:

1. `spec_clarifications.status = OPEN`
2. Project is Tier 2 or Tier 3
3. `active_role = CLIENT`, `client_subtype = TECH_TEAM`

Main Success Scenario:

Same as UC05 Steps 1–5 - TECH_TEAM responds with technical authority on architecture-related components.

Postconditions: Same as UC05.

UC06a - Review Capability Bids - Seam Analysis; Flag Recommended / Not Recommended

Primary Actor: CLIENT / TECH_TEAM

Feature reference: F6 Surface B (entry), bid state machine

Preconditions:

1. `bid.state = SUBMITTED` → NestJS auto-advances to `TECH_REVIEW` when TECH_TEAM opens bid
2. `active_role = CLIENT`, `client_subtype = TECH_TEAM`
3. Artifact B accessible (engagement `CONNECTED` state not required at this stage - TECH_TEAM reviews bids before connection)

Main Success Scenario:

1. TECH_TEAM receives notification: "A new bid has been submitted for your review"

2. Opens bid review panel: sees bid's footprint alignment statement, architectural approach, milestone pricing, seam gap map
3. Reviews against their knowledge of actual system architecture (only they can assess alignment accuracy)
4. [IF satisfied] Flags bid as "Recommended" → `bid.state` → `TECH_APPROVED`; CEO_REVIEW unlocked
5. [IF concerns] Requests bid revision (see `UC06r`) OR flags "Not Recommended" with required reason text → `bid.state` → `TECH_DISAPPROVED`; Surface D triggered for CEO

Extensions:

- `<<extend>>` `UC06r` - TECH_TEAM can request bid component revision before making final recommendation

Postconditions:

- `bid.state` = `TECH_APPROVED` or `TECH_DISAPPROVED`
 - If `TECH_APPROVED`: CEO notified; CEO_REVIEW unlocked
 - If `TECH_DISAPPROVED`: CEO notified; Surface D gate activates before CEO_REVIEW
-

UC06r - Request Bid Component Revision (Surface B)

Primary Actor: CLIENT / TECH_TEAM

Feature reference: F6 Surface B

Extends: UC06a at revision request decision point

Preconditions:

1. `bid.state` = `TECH_REVIEW`
2. `active_role` = `CLIENT`, `client_subtype` = `TECH_TEAM`

Main Success Scenario:

1. TECH_TEAM identifies a specific bid component requiring revision (footprint alignment / architectural approach / milestone pricing)
2. Flags the component with a structured reason: `{ flagged_component, reason }` → `bid_revision_requests` row written
3. `bid.state` → `REVISION_REQUESTED`; expert notified
4. Expert revises only the flagged component (other components locked) via `UC20r`
5. `bid.state` → `REVISED`; `bid_versions` immutable row written
6. Loop back to `UC06a` TECH_REVIEW - unlimited rounds permitted

Postconditions:

- `bid_versions` row written (immutable; RQ2 audit trail)
 - `bid.state` = `REVISED`; TECH_TEAM review loop continues
-

UC07b - Access Artifact B Post-Connection (TECH_TEAM Only)

Primary Actor: CLIENT / TECH_TEAM

Feature reference: F5

Preconditions:

1. `engagement.state ≥ CONNECTED`
2. `active_role = CLIENT, client_subtype = TECH_TEAM`
3. NDA accepted by CEO (engagement-level flag)

Main Success Scenario:

1. TECH_TEAM opens Tech Dashboard; "Project Blueprint" panel now active (was locked before connection)
2. System DB-level check: `engagement.state ≥ CONNECTED` → returns Artifact B fields (schemas, payload samples, integration contracts, sensitive uploads from Stage 4)
3. Pay-gated documents (if released by milestone funding) also visible in document inbox
4. TECH_TEAM reads technical blueprint; can reference during bid review and sprint sign-offs

Guard note: CEO `active_role = CLIENT/CEO` attempting to query Artifact B fields receives HTTP 403 regardless of engagement state - DB-level hard guard, not a UI hide.

Postconditions: No state change - read-only access.

UC08t - Sign Off Technical Milestones Criterion by Criterion

Primary Actor: CLIENT / TECH_TEAM

Feature reference: F7 Layer 1

Preconditions:

1. `milestone.state = SUBMITTED`
2. `milestone.sign_off_authority = TECH_TEAM` or `JOINT`
3. `active_role = CLIENT, client_subtype = TECH_TEAM`

Main Success Scenario:

1. TECH_TEAM notified: "Expert has submitted Milestone {n} for technical review"
2. Opens milestone review panel; sees each acceptance criterion with deliverable evidence
3. Verifies criteria one by one: marks `verified_at + verifier_role = TECH_TEAM`
4. For JOINT milestones: both TECH_TEAM and CEO must verify their respective criteria before final APPROVED transition is permitted

Extensions:

- [IF criterion not met] TECH_TEAM marks criterion as failed + writes `revision_requests` record; `milestone.state → IN_REVISION`
- [IF dispute required] `milestone.state → DISPUTED`; TECH_TEAM can file dispute from SUBMITTED or IN_REVISION state

Postconditions:

- Technical criteria marked `verified_at`
 - For TECH_TEAM-only milestones: `milestone.state` → `APPROVED` after final criterion
 - For JOINT milestones: awaits CEO sign-off to complete
-

UC10t - Approve Add-On Phase - Technical Justification Dimension

Primary Actor: CLIENT / TECH_TEAM

Feature reference: F8

Preconditions:

1. `addon_phase_requests.state` = `PENDING_TECH_APPROVAL`
2. `active_role` = `CLIENT`, `client_subtype` = `TECH_TEAM`

Main Success Scenario:

1. TECH_TEAM notified: "Expert has submitted an Add-On Phase Brief for your technical review"
2. Opens brief; reads causal chain, new capability requirement domain/seam assertions, why invisible at intake
3. TECH_TEAM evaluates: does the causal chain technically hold up? Does the new capability claim make sense given the actual system architecture?
4. Approves; `addon_phase_requests.tech_team_approval` = `APPROVED`; state → `PENDING_CEO_APPROVAL`
5. CEO notified; **UC10** triggered

Extensions:

- [IF TECH_TEAM rejects] Brief returned to expert with technical objection note; expert revises and resubmits

Postconditions:

- `addon_phase_requests.tech_team_approval` = `APPROVED`
 - CEO review unlocked
-

UC12b - Complete Post-Engagement Review (Tech Team Structured Seam Signal Form)

Primary Actor: CLIENT / TECH_TEAM

Feature reference: F11

Preconditions:

1. `engagement.state` = `CLOSED`
2. TECH_TEAM has not yet submitted a review for this engagement

Main Success Scenario:

1. TECH_TEAM opens structured review form: Overall rating (1–5), seam-specific performance questions (ground truth proactivity, scope accuracy, add-on phase causal chain quality), open text comment
2. Submits; `reviews` row written { `reviewer_role: TECH_TEAM` }
3. Seam-specific answers written to `expert_seam_outcome_signals` table; Tier 4 signal accumulation rule evaluated by NestJS against the new signal

Postconditions:

- `reviews` row written; `expert_seam_outcome_signals` rows written
- Tier 4 signal accumulation evaluated; if threshold met → expert seam tier auto-upgraded to Tier 4 (no admin action)

UC13t - Read Sprint Plan and DoD Checklist; Comment on Sprint Thread

Primary Actor: CLIENT / TECH_TEAM

Feature reference: F7 Layer 2, Layer 3

Preconditions:

1. `engagement.state = ACTIVE` and `milestone.state = IN_PROGRESS`
2. Sprint plan created by expert (UC23)

Main Success Scenario:

1. TECH_TEAM opens sprint dashboard: sees sprint task list, task statuses (TODO / IN_PROGRESS / DONE), DoD checklist (read-only), sprint status updates submitted by expert
2. Can leave a comment per sprint thread (lightweight; separate from main messaging)
3. TECH_TEAM cannot modify sprint plan or DoD items

Postconditions: No state change - read/comment only.

PART C - EXPERT FLOWS

UC13 - Create Taxonomy-Based Profile and Seam Declarations

Primary Actor: EXPERT

Feature reference: F3

Subscription gate: Free tier - profile creation available without Pro

Preconditions:

1. `active_role = EXPERT`
2. Account registered

Main Success Scenario:

1. Actor opens Profile Setup: six capability domain declarations (Deep / Working / Aware per domain)
2. Actor declares seam claims (self-declared Tier 1 for each claimed seam)
3. Actor declares engagement model (Advisory only / Spec + review / Full implementation)
4. Actor adds stack tags (multi-select)
5. Actor adds archetype history (self-declared initially)
6. Actor saves; `expert_profiles` and `seam` rows created; all seams at `verification_tier = 1` (Claimed)

Extensions:

- `<<extend>>` UC15 - Actor can immediately proceed to portfolio evidence submission for Tier 2 upgrade
- `<<extend>>` UC16 - Actor can immediately request scenario assessment for Tier 3 upgrade

Postconditions:

- `expert_profiles` row created
- `seam` rows created with `verification_tier = 1`
- Expert discoverable in matching engine (at Claimed tier weight 0.20)

UC14 - Purchase Expert Pro Subscription from Wallet Balance

Primary Actor: EXPERT

Feature reference: F1.5

Subscription gate: This UC activates the gate

Preconditions:

1. `active_role = EXPERT`
2. `subscription_expert_tier = 'free'`
3. `wallet.available_balance >= 300,000 VND` (Expert Pro price)

Main Success Scenario:

Same flow as UC03 but for Expert role: price = 300K VND; sets `subscription_expert_tier = 'pro'`; unlocks LLM portfolio evidence verification, scenario assessment, Tier 2+ project bidding, AI Service Generator, and earnings analytics dashboard.

Includes:

- `<<include>>` UC02 - called as prerequisite redirect if balance insufficient

Postconditions:

- `users.subscription_expert_tier = 'pro'`
- UC15, UC16, UC18 (AI generator), UC20 on Tier 2+ projects now accessible

UC15 - Submit Portfolio Evidence for LLM Auto-Verification (Tier 2)

Primary Actor: EXPERT

Feature reference: F3, Expert Verification Tier 2

Subscription gate: Expert Pro required

Preconditions:

1. `active_role = EXPERT; subscription_expert_tier = 'pro'`
2. Target seam exists in expert's profile at Tier 1 (Claimed)
3. `seam.lockout_active = false` (not in 30-day lockout from 5-attempt limit)

Main Success Scenario:

1. Actor selects a seam for Tier 2 verification; opens Portfolio Evidence form
2. Form presents structured decision-point questions (3 required):
 - What was the system's architecture when you entered the engagement?
 - What were the 2–3 most consequential technical decisions you made?
 - What would have happened if you had made a different choice at each point?
3. Actor writes responses (sanitization instruction shown: no client names, proprietary code, NDA-protected details)
4. Actor submits; FastAPI LLM extraction runs: checks for all required signal types for this seam at ≥ 0.85 confidence
5. [IF extraction confidence ≥ 0.85 AND all required signal types found] Auto-upgrade:
`seam.verification_tier = 2; PlatformDecision` row written; actor notified immediately
6. [IF confidence below threshold OR any required signal type missing] LLM generates specific gap advisory (naming which signal types were found vs. missing); submission returned;
`seam.attempt_count += 1`

Extensions:

- [IF `seam.attempt_count = 5`] `seam.lockout_active = true; lockout_until = now + 30 days`; actor notified of lockout

Postconditions (success):

- `seam.verification_tier = 2`; confidence factor in composite score = 0.55
- `PlatformDecision` row persisted (visible in Admin Seam Verification Log)

UC16 - Request and Complete Scenario Assessment (Tier 3 Auto-Delivery + LLM Auto-Grading)

Primary Actor: EXPERT

Feature reference: F3, Expert Verification Tier 3

Subscription gate: Expert Pro required

Preconditions:

1. `active_role = EXPERT; subscription_expert_tier = 'pro'`
2. Target seam at `verification_tier = 2` (Tier 3 requires Tier 2 first)
3. `seam.lockout_active = false`

Main Success Scenario:

1. Actor requests Tier 3 verification for a seam
2. System auto-selects a question from the question bank (lowest `last_used_at` for this seam - prevents gaming)
3. Actor receives scenario question; writes response (open text)
4. Actor submits; FastAPI LLM evaluates response against stored rubric for that question: all required dimensions must individually pass (not an average)
5. [IF all dimensions pass] Auto-upgrade: `seam.verification_tier = 3`; `PlatformDecision` row written; actor notified
6. [IF any dimension fails] LLM generates specific failure note (names which dimension failed and what it needed); new question auto-selected for resubmission; `seam.attempt_count += 1`

Extensions:

- [IF `seam.attempt_count = 5`] 30-day lockout same as UC15

Postconditions (success):

- `seam.verification_tier = 3`; confidence factor = 0.80
- Expert competitive for high-tier projects requiring verified seam coverage

UC17 - Link Bank Account via Bank Hub Hosted Link

Primary Actor: EXPERT

Feature reference: F10 Bank Hub integration

Preconditions:

1. `active_role = EXPERT`
2. `users.sepay_bank_account_xid` is null (bank not yet linked)

Main Success Scenario:

1. Actor opens "Link Bank Account" in Expert Dashboard (prompted on first withdrawal attempt, or proactively on registration)
2. NestJS calls Bank Hub Link Token API: `POST /bankhub/v1/link-tokens { user_id, purpose: "LINK_BANK_ACCOUNT" }`; receives `hosted_link_url`
3. System embeds hosted link as iframe/WebView in Expert Dashboard
4. Actor selects bank, enters account number, enters OTP (bank-verified; no password sharing)
5. Bank Hub fires `BANK_ACCOUNT_LINKED` webhook: `{ bank_account_xid, account_number, account_holder_name, brand_name }`
6. NestJS handler sets: `users.sepay_bank_account_xid = bank_account_xid`;
`bank_account_holder_name = account_holder_name` (bank-verified); `bank_linked_at = now()`
7. Actor notified: "Bank account linked: {bank_name} | {account_number}"

Postconditions:

- `users.sepay_bank_account_xid` set (bank-verified; not self-reported)
 - `UC27` (withdrawal) now accessible
-

UC18 - Publish AI Service Listing via AI Generator (Path B Marketplace)

Primary Actor: EXPERT

Feature reference: F3 Path B, AI Service Generator

Subscription gate: Expert Pro required (for AI generator route); Expert Free can publish manually

Preconditions:

1. `active_role = EXPERT`
2. Expert profile exists (`UC13` completed)

Main Success Scenario:

1. Actor opens "Create Service Listing"
2. [Optional AI path - requires Expert Pro] Actor inputs key capabilities and target use cases in plain text; FastAPI LLM generates structured description (title, description, what's included, engagement model, timeline estimate, suggested price range)
3. Actor reviews and edits LLM-generated description (or writes manually)
4. Actor sets final price (VND), engagement type (SERVICE_PURCHASE or TECH_DISCOVERY), availability
5. Actor publishes; `service_listings` row created { `state: ACTIVE` }
6. Listing appears in Path B marketplace; discoverable by CEO browsing `UC30`

Extensions:

- [IF LLM generator content fails content moderation] Rejected; LLM rejection logged in `PlatformDecision`; actor prompted to revise

Postconditions:

- `service_listings.state = ACTIVE`
 - Listing visible in Path B marketplace
-

UC19 - Browse Project Shortlist → File Spec Clarification Request Before Bidding (Surface A)

Primary Actor: EXPERT

Feature reference: F6 Surface A

Subscription gate: Expert Pro (to view bid invitations for Tier 2+ projects)

Preconditions:

1. `active_role = EXPERT`
 2. Expert appears in `match_shortlist` for a project (match score threshold met)
 3. `project.spec_state = PUBLISHED`
-

Main Success Scenario:

1. Actor receives bid invitation; views Artifact A and their personal seam gap map
2. [Optional before bidding] Actor files one or more spec clarification requests: `POST /specs/{spec_id}/clarifications { target_component, question_text }`
3. Multiple clarifications can be OPEN simultaneously; state = OPEN
4. Actor receives responses from TECH_TEAM (UC05t) or CEO (UC05); marks each RESOLVED
5. [IF proceeding to bid with unresolved clarifications] Bid form flags unresolved clarifications with a warning; actor can still submit

Extensions:

- `<<extend>>` UC20 - after clarifications resolved (or waived), actor proceeds to submit bid

Postconditions:

- `spec_clarifications` rows written; status = OPEN until answered and resolved

UC20 - Submit Structured Capability Bid (3 Required Components)

Primary Actor: EXPERT

Feature reference: F6 bid submission

Subscription gate: Expert Pro (for Tier 2+ project bids)

Preconditions:

1. `active_role = EXPERT`; `subscription_expert_tier = 'pro'` for Tier 2+ projects
2. Expert in project's `match_shortlist`
3. `project.spec_state = PUBLISHED`

Main Success Scenario:

1. Actor opens bid form; sees footprint alignment section pre-populated from their expert profile
2. **Component 1 - Footprint Alignment Statement:** Actor confirms domain depth alignment and seam claims; any claimed-only seam on a load-bearing position flagged: *"This seam is load-bearing. Your Tier 1 claim will be visible to the client. Would you like to complete the scenario assessment first?"*
3. **Component 2 - Architectural Approach Summary:** Actor writes approach addressing SDLC milestone framework from Artifact A and any stated-approach discrepancy; must not include proprietary design (no Artifact B yet)
4. **Component 3 - Conditional Milestone Pricing:** Actor enters pricing per milestone; each conditional requires: `{ condition, price_range_min_vnd, price_range_max_vnd, trigger_description }`; free-text "TBD" rejected (422)
5. Actor submits; NestJS validates all 3 components present (422 if any missing); `bid.state` → SUBMITTED

Extensions:

- `<<extend>>` UC19 - Actor may have filed spec clarifications before reaching this UC

- <<extend>> UC20r - bid may be returned for component revision (Surface B)
- <<extend>> UC20c - price negotiation may be initiated by CEO after TECH_APPROVED

Postconditions:

- bids row created; bid.state = SUBMITTED
- TECH_TEAM notified; UC06a triggered

UC20r - Revise Specific Bid Component on REVISION_REQUESTED (Surface B)

Primary Actor: EXPERT

Feature reference: F6 Surface B

Extends: UC20 at REVISION_REQUESTED state

Preconditions:

1. bid.state = REVISION_REQUESTED
2. bid_revision_requests row exists with flagged_component identified
3. active_role = EXPERT

Main Success Scenario:

1. Actor receives notification: "TECH_TEAM has requested revision of [flagged_component] - reason: [text]"
2. Actor opens bid form; only the flagged component is editable; all other components locked
3. Actor updates the flagged component
4. Submits revision; NestJS: bid_versions immutable row written (version_number incremented, snapshot of old component stored); bid.state → REVISED
5. TECH_TEAM notified; loop returns to UC06a TECH_REVIEW

Postconditions:

- bid_versions row written (immutable; feeds RQ2 research data on spec evolution)
- bid.state = REVISED

UC20c - Accept / Counter / Decline Price Negotiation Proposal (Surface C)

Primary Actor: EXPERT

Feature reference: F6 Surface C

Extends: UC20 at price negotiation step; paired with UC06d (CEO side)

Preconditions:

1. bid.state = PRICE_NEGOTIATION
2. price_negotiations row exists with CEO's proposal
3. active_role = EXPERT

Main Success Scenario:

1. Actor receives notification: "CEO has proposed a price adjustment"
2. Actor reviews proposed per-milestone pricing vs. their original pricing
3. Actor chooses: ACCEPT → `bid.state` → `NEGOTIATION_COMPLETE`; CEO notified; or COUNTER (if round < 2) → `price_negotiations` counter row written; CEO notified; or DECLINE → negotiation ends; bid returns to original pricing for CEO_REVIEW
4. [IF actor COUNTERs on round 2] CEO receives final counter; CEO must ACCEPT or DECLINE (no further counter; NestJS returns 422 on CEO counter attempt)

Postconditions:

- `price_negotiations` rows updated
 - If ACCEPT: final pricing locked; `bid.state` = `NEGOTIATION_COMPLETE`
 - If DECLINE: negotiation closed; CEO can proceed with original pricing
-

UC21 - Accept Connection and Access Artifact B

Primary Actor: EXPERT

Feature reference: F5

Preconditions:

1. `engagement.state` = `PENDING` (connection request sent by CEO via UC07)
2. `active_role` = `EXPERT`

Main Success Scenario:

1. Actor receives connection request notification; views Artifact A (Artifact B still locked)
2. Actor reviews Artifact A fully; decides to accept
3. Actor accepts; `engagement.state` → `CONNECTED`
4. NDA click-through presented; actor acknowledges (checkbox)
5. Artifact B `access_state` → `UNLOCKED`; actor can now query Artifact B fields
6. Actor reviews Artifact B: tech team's schemas, payload samples, integration contracts, sensitive uploads

Extensions:

- [IF actor declines] `engagement.state` → `DECLINED`; CEO notified; can select another expert

Postconditions:

- `engagement.state` = `CONNECTED`
 - Actor can access Artifact B
 - Messaging thread active (UC31)
-

UC22 - Stage Pay-Gated Reasoning Documents with Milestone Release Trigger

Primary Actor: EXPERT

Feature reference: F5 Pay-gated knowledge staging

Preconditions:

1. `engagement.state ≥ CONNECTED`
2. `active_role = EXPERT`
3. Milestone exists with `state = DEFINED` or later

Main Success Scenario:

1. Actor opens staging panel; uploads architecture design rationale document(s)
2. Actor tags each document with a milestone release trigger: "Release this document to the client's tech team when Milestone {n} payment is confirmed"
3. Document staged with `release_state = STAGED`
4. When SePay IPN confirms milestone funding (**UC08** success): NestJS automatically sets `release_state → RELEASED`; document becomes visible in TECH_TEAM's document inbox
5. TECH_TEAM receives unfiltered design rationale; CEO cannot access these documents

Postconditions:

- Staged documents released to TECH_TEAM automatically on milestone funding
 - Expert's full design reasoning disclosed post-commitment (IP deadlock resolution)
-

UC23 - Create Sprint Plan + DoD Checklist for Funded Milestone

Primary Actor: EXPERT

Feature reference: F7 Layers 2 & 3

Preconditions:

1. `milestone.state = IN_PROGRESS` (funded; IPN confirmed)
2. `active_role = EXPERT`

Main Success Scenario:

1. Actor opens Milestone Management panel; sees "Create Sprint Plan" prompt
2. Actor creates sprint plan: sprint number, week range, deliverable checkpoint, task list per sprint (TODO items)
3. Actor creates DoD checklist: list of required and non-required items; each required item has `is_required = true`
4. [Optional] LLM criterion quality check on acceptance criteria fires on save: flags subjective language ("looks good"); actor can override with acknowledgment
5. Saves; `milestone_sprints` and `milestone_dod_items` rows created

Postconditions:

- `milestone_sprints` rows created
 - `milestone_dod_items` rows created with `status = PENDING`
-

- TECH_TEAM can now view sprint plan and DoD (read-only)

UC24 - Submit Weekly Sprint Status Update (ON_TRACK / DEVIATION / BLOCKER)

Primary Actor: EXPERT

Feature reference: F7 Layer 3

Preconditions:

1. `milestone.state = IN_PROGRESS`
2. Sprint plan exists (UC23 completed)
3. `active_role = EXPERT`

Main Success Scenario:

1. Actor opens Sprint Status form (due on configurable day, default Monday)
2. Actor selects status: `ON_TRACK | DEVIATION | BLOCKER`
3. [IF `ON_TRACK`] Submits; sprint dashboard updated; no further action
4. [IF `DEVIATION`] Required: `deviation_note` (adjusted timeline + reason); CEO and TECH_TEAM notified; milestone clock unchanged
5. [IF `BLOCKER`] Required: `blocker_type` (`SCOPE_GAP | TECH_CONSTRAINT | EXTERNAL_DEPENDENCY | SCOPE_EVOLUTION`) + `blocker_description` + `estimated_resolution`
6. [IF `blocker_type = SCOPE_EVOLUTION`] `scope_evolution_flag = true`; F8 Add-On Phase Protocol auto-triggered; milestone clock paused

Extensions:

- [IF overdue by 3 days] System sends reminder to expert
- [IF overdue by 5 days] CEO and TECH_TEAM notified: "Expert has not submitted this week's status update"
- `<<extend>>` UC26 - `SCOPE_EVOLUTION` flag auto-triggers Add-On Phase Brief submission

Postconditions:

- `sprint_status_updates` row written
- If `SCOPE_EVOLUTION`: `addon_phase_requests` initiated; milestone clock paused

UC25 - Submit Milestone Deliverable (DoD Gate Enforced at Route Level)

Primary Actor: EXPERT

Feature reference: F7 Layer 1 & 2

Preconditions:

1. `milestone.state = IN_PROGRESS`

2. All `is_required = true` DoD items have `status = COMPLETED` with `completion_note` set (route-level guard)
3. `active_role = EXPERT`

Main Success Scenario:

1. Actor marks all required DoD items as `COMPLETED` with completion notes; non-required items marked `COMPLETED` or `NOT_APPLICABLE` with notes
2. Actor clicks "Submit Milestone Deliverable"
3. NestJS guard: `WHERE milestone_id = ? AND is_required = true AND status != 'COMPLETED' → if any rows returned → 422 { code: "DOD_INCOMPLETE", unchecked_items: [...] }`; submission blocked
4. [IF all required items `COMPLETED`] `milestone.state → SUBMITTED`; sign-off authority notified; review clock starts

Extensions:

- [IF `TECH_TEAM` or CEO disputes within review window] `milestone.state → DISPUTED`

Postconditions:

- `milestone.state = SUBMITTED`
- Sign-off authority (`TECH_TEAM` or CEO per `sign_off_authority` field) notified

UC26 - Submit Add-On Phase Brief with Causal Chain

Primary Actor: EXPERT

Feature reference: F8

Extends: UC24 at `SCOPE_EVOLUTION` extension point (auto-triggered)

Preconditions:

1. `sprint_status_updates.scope_evolution_flag = true` (from UC24)
2. `milestone.state = IN_PROGRESS` (clock paused)
3. `active_role = EXPERT`

Main Success Scenario:

1. Actor opens Add-On Phase Brief form (auto-presented from `SCOPE_EVOLUTION` sprint status)
2. Actor fills three required fields:
 - *Causal chain*: What did completed phases produce that made this new requirement visible? (free text - explains the discovery was a consequence of work, not a scoping error)
 - *New capability requirement*: Which domains and seams does the new work require that were absent from the original footprint?
 - *Why it was invisible at intake*: Explicit acknowledgment that this is a designed lifecycle event
3. Actor submits; `addon_phase_requests` row created; routed to `TECH_TEAM` (UC10t) and CEO (UC10) for dual approval
4. Admin receives informational log entry (not an approval gate)

Postconditions:

- `addon_phase_requests` row created; `state = PENDING_TECH_APPROVAL`
 - Milestone clock paused until both approvals received
-

UC27 - Request Withdrawal (Chi Hộ Fires Automatically)

Primary Actor: EXPERT

Feature reference: F10 Withdrawal flow

Preconditions:

1. `active_role = EXPERT`
2. `users.sepay_bank_account_xid` IS NOT NULL (UC17 completed)
3. `wallet.available_balance >= requested_amount` AND `>= 100,000 VND` minimum

Main Success Scenario:

1. Actor opens Wallet panel; clicks "Withdraw"; enters `amount_vnd`
2. NestJS guard validates bank account linked and balance sufficient
3. Atomic DB transaction: `wallet.available_balance -= amount`; `wallet_transactions { type: WITHDRAWAL }; withdrawal_requests { status: PENDING }`
4. After commit (async): NestJS calls chi hộ API: `POST /chiho/v1/disburse { amount, bank_account_xid, reference: "WD-{id}" }`
5. SePay processes (seconds to minutes); credit IPN fires on expert's linked bank account
6. NestJS IPN handler: `withdrawal_requests.status → COMPLETED`; `confirmed_at = now()`; actor notified: "Withdrawal of {amount} VND completed to {bank_name}"

Extensions:

- [IF chi hộ API returns error] `withdrawal_requests.status → FAILED`; `wallet.available_balance += amount` (atomic rollback); actor notified: "Withdrawal failed. Balance restored."

Includes:

- `<<include>> UC17` - bank account must be linked (called as prerequisite redirect if `bank_account_xid` is null)

Postconditions (success):

- `withdrawal_requests.status = COMPLETED`
 - Funds transferred to expert's verified linked bank account
 - Zero admin involvement
-

UC28 - Complete Post-Engagement Review (Expert Form)

Primary Actor: EXPERT

Feature reference: F11

Preconditions:

1. `engagement.state = CLOSED`
2. Expert has not yet submitted a review for this engagement
3. `active_role = EXPERT`

Main Success Scenario:

1. Actor opens review form: assesses both client sub-roles together (CEO + TECH_TEAM experienced as a unit)
2. Fills: Overall rating (1–5), Milestone approval responsiveness (sign-off timeliness), Technical information availability ("Was Artifact B complete and accurate when you first accessed it?"), CEO communication clarity, Open text
3. Submits; `reviews` row written `{ reviewer_role: EXPERT }`
4. Responses contribute to client-side reputation (approval cycle time averages tracked per role)

Postconditions:

- `reviews` row written
- Client reputation signals updated

PART D - DUAL-ROLE FLOWS

UC-DR1 - Add Second Role to Account

Primary Actor: CLIENT / CEO (adding EXPERT role) or EXPERT (adding CLIENT_CEO role)

Feature reference: F1 dual-role

Preconditions:

1. Actor is authenticated; `roles` array has only one role
2. Actor wants to hold both CLIENT_CEO and EXPERT roles on the same account

Main Success Scenario:

1. Actor opens "Account Settings → Add Role"
2. System presents verification step (identity confirmation)
3. On success: `users.roles` array updated to `["CLIENT_CEO", "EXPERT"]`
4. Role switcher appears in persistent top nav
5. Two separate `user_subscriptions` rows may exist - one per role type; both deduct from same wallet

Postconditions:

- `users.roles = ["CLIENT_CEO", "EXPERT"]`
- Role switcher enabled

- Self-exclusion guard active: matching engine filters `expert.user_id NOT IN project.involved_parties` regardless of active role
-

UC-DR2 - Switch Active Role via Role Switcher

Primary Actor: Dual-role user (CLIENT_CEO + EXPERT)

Feature reference: F1 role switcher

Preconditions:

1. `users.roles.length > 1`
2. Role switcher visible in top nav

Main Success Scenario:

1. Actor clicks role switcher toggle in persistent top nav
2. NestJS reissues JWT with `active_role` updated to selected role
3. React re-renders dashboard for new role context: CEO dashboard ↔ Expert dashboard
4. No re-login required

Postconditions:

- New JWT issued with updated `active_role`
 - Dashboard context changed; all guards re-evaluate against new role
-

UC-DR3 - Fund Project Milestone from Expert Earnings (Same Wallet)

Primary Actor: Dual-role user acting as CLIENT_CEO

Feature reference: F1, F10

Preconditions:

1. User holds both CLIENT_CEO and EXPERT roles
2. `active_role = CLIENT` (CEO mode)
3. Expert earnings credited to `wallet.available_balance` from past engagements
4. Milestone awaiting funding (`milestone.state = AWAITING_PAYMENT`)

Main Success Scenario:

1. Actor switches to CEO role via `UC-DR2`
2. Wallet balance shows combined available balance (expert earnings + any CEO top-ups)
3. Actor funds milestone via `UC08` - escrow lock deducted from same wallet
4. No external top-up needed if earnings are sufficient

Postconditions:

- Milestone funded from internal wallet balance (no external transfer needed)
 - This is the designed "frictionless re-investment" scenario for dual-role users
-
-

PART E - ADMIN FLOWS

UC-A1 - Emergency Spec Pull-Back

Primary Actor: ADMIN

Feature reference: F12 Module 1 / Module 3, F2 spec state machine

Preconditions:

1. `active_role = ADMIN`
2. `project.spec_state = PUBLISHED`
3. Emergency condition identified (factual error reported in published spec, or elicitation failure pattern detected)

Main Success Scenario:

1. Admin opens Spec Emergency Queue or Platform Integrity Monitor
2. Admin identifies the spec requiring pull-back; reads the reason (e.g., expert-reported factual error in Artifact A)
3. Admin enters a reason note (visible to client; not to experts currently shortlisted)
4. Admin confirms pull-back: `project.spec_state → SUSPENDED`
5. Artifact A visibility removed from expert shortlist views
6. Client notified; can correct spec and re-enter elicitation engine

Guard note: This is the ONLY write action in Module 1 / Module 3. Admin cannot pull back DRAFT or RETURNED_TO_CLIENT specs - only PUBLISHED ones.

Postconditions:

- `spec_state = SUSPENDED`
- Spec removed from matching engine and expert views
- `PlatformDecision` row written with admin reason

UC-A2 - Monitor Platform Integrity Monitor

Primary Actor: ADMIN

Feature reference: F12 Module 1

Preconditions:

1. `active_role = ADMIN`

Main Success Scenario:

1. Admin opens Platform Integrity Monitor; reads four sub-logs:
 - *Spec auto-return log:* failed specs, void type, LLM advisory note sent, re-entry point

- *Seam verification log*: all auto-upgrades (Tier 1→2, 2→3) with LLM confidence; all auto-returns with gap advisory; lockout events
- *Tier 4 elevation log*: automatic Tier 4 upgrades from signal accumulation (seam, engagement, signal type, outcome score)
- *Dispute resolution log*: all disputes, resolution layer used, LLM confidence for Layer 1, "Report this resolution" feedback

Extensions:

- [IF spec pull-back required from this view] <<extend>> UC-A1
- [IF account suspension required from this view] <<extend>> UC-A2 calls account suspension action (see F12 Module 2)

Postconditions: No state change - read-only monitoring.

UC-A3 - Monitor Transaction Ledger

Primary Actor: ADMIN

Feature reference: F12 Module 5

Preconditions:

1. `active_role = ADMIN`

Main Success Scenario:

1. Admin opens Transaction Ledger module
2. Views full `wallet_transactions` ledger with SePay transaction IDs, milestone references, timestamps
3. Views `withdrawal_requests` audit trail: status, Telegram notification timestamps, chi hộ confirmation timestamps, matched amounts
4. Filters by date range, user, engagement, transaction type
5. Exports data for research (RQ1, RQ2, RQ3 evidence)

Postconditions: No state change - read/export only.

UC-A4 - Monitor Bid Conflict Override Log (Surface D Decisions)

Primary Actor: ADMIN

Feature reference: F12 Module 1, F6 Surface D

Preconditions:

1. `active_role = ADMIN`

Main Success Scenario:

1. Admin opens Platform Integrity Monitor; filters `PlatformDecision` log for `decision_type = CONFLICT_OVERRIDE`

2. Reviews: which bid, which project, CEO's override reason, TECH_TEAM's disapproval reason, post-engagement outcome (if engagement closed)
3. Correlates override records with negative engagement outcomes → RQ3 evidence on trust factor impact

Postconditions: No state change - read-only audit.

UC-A5 - Monitor Platform Analytics and Export Research Data

Primary Actor: ADMIN

Feature reference: F12 Module 6

Preconditions:

1. `active_role = ADMIN`

Main Success Scenario:

1. Admin opens Analytics Dashboard; views:
 - Active projects by archetype and tier
 - Average composite matching score for connected engagements vs. declined bids (RQ1 evidence)
 - Elicitation completion rate and auto-publish pass rate (RQ2 evidence)
 - Seam verification throughput: auto-upgrade rate, auto-return rate, resubmissions before upgrade
 - Milestone completion rate, average revision cycles
 - Dispute rate, Layer 1 auto-resolution rate, average time-to-resolution by layer (RQ3 trust evidence)
 - Review completion rate, average ratings
2. Exports filtered datasets as CSV for final report

Postconditions: No state change - read/export only.

PART F - GENERAL FLOWS

UC29 - File and Resolve Dispute (3-Layer Engine)

Primary Actor: CLIENT / CEO, CLIENT / TECH_TEAM, or EXPERT

Secondary Actor: System (LLM - FastAPI)

Feature reference: F12 / Dispute Escrow Automation

Extends: UC09 or UC25 (from SUBMITTED or IN_REVISION states)

Preconditions:

1. `milestone.state = SUBMITTED` or `IN_REVISION`
 2. Actor is part of the active engagement
-

Main Success Scenario:

1. Actor clicks "File Dispute"; inputs reason and desired outcome.
2. NestJS atomic transaction: `milestone.state` → `DISPUTED`; `escrow_accounts.status` → `FROZEN`.
3. System triggers Layer 1 (AI Resolution): FastAPI LLM evaluates deliverable against acceptance criteria.
4. [IF LLM Confidence ≥ 0.80] System auto-resolves. `escrow_accounts.status` → `RELEASED` (if Expert wins) or `REFUNDED` (if Client wins). Ledger updated automatically.
5. [IF LLM Confidence < 0.80] System triggers Layer 2: Starts 48-hour cooling timer and sends Mutual Agreement form to both parties.
6. [IF both parties agree on form] System executes mutually agreed split.
7. [IF 48 hours expire with no agreement] System triggers Layer 3: Automatic 50/50 ledger split. Expert's 50% triggers Chi Hộ withdrawal; Client's 50% refunded to wallet.
8. `PlatformDecision` audit row written.

Postconditions:

- Milestone escrow resolved (Released, Refunded, or Split).
- Milestone state advanced to CLOSED/RESOLVED.

UC30 - Browse Path B Expert Marketplace

Primary Actor: CLIENT / CEO or CLIENT / TECH_TEAM

Feature reference: F3 Path B

Subscription gate: Free tier - browsing requires no subscription

Preconditions:

1. `active_role` = `CLIENT` (CEO or TECH_TEAM)

Main Success Scenario:

1. Actor opens Path B Marketplace
2. Sees service cards: title, domains, seam tags, engagement model, price, expert reputation (rating + completion count)
3. Filters by: domain, seam, archetype, price range, availability
4. Clicks into a service card for full detail view

Extensions:

- [EXTEND: `UC11`] CEO can purchase a service directly from this view (TECH_TEAM browse-only; cannot purchase)

Postconditions: No state change - read-only.

UC31 - Real-Time Messaging Within Engagement

Primary Actor: CLIENT / CEO, CLIENT / TECH_TEAM, EXPERT (all three transactional roles share one thread); ADMIN (read-only)

Feature reference: F9

Preconditions:

1. `engagement.state ≥ CONNECTED` (messaging thread created on connection)
2. Actor is one of the three transactional roles linked to this engagement OR `active_role = ADMIN`

Main Success Scenario:

1. Actor opens Engagement Messaging panel; sees shared thread with all three transactional roles
2. Actor types message; sends
3. Socket.io delivers message in real time to all active participants
4. Message persisted in PostgreSQL `messages` table
5. Unread badge updated for other participants

Extensions:

- File attachments supported (milestone deliverable sharing alternative)
- ADMIN can view all threads (read-only) for dispute audit; cannot send messages

Postconditions:

- `messages` row written to PostgreSQL
 - All engagement participants notified
-

UC32 - View Wallet, Transaction History, Subscription Status

Primary Actor: All authenticated roles (CLIENT / CEO, CLIENT / TECH_TEAM, EXPERT, ADMIN - each sees a role-specific panel)

Feature reference: F10 Transaction history, F1.5 Subscription

Preconditions:

1. Actor is authenticated

Main Success Scenario - CEO panel:

1. Actor sees: wallet available balance, locked balance; funded milestones with VA numbers and amounts; wallet top-up history; total project spend; subscription tier + expiry date
2. "Top Up" button → UC02; "Upgrade Subscription" → UC03

Main Success Scenario - TECH_TEAM panel:

1. Actor sees: milestone status and release dates; pay-gated document inbox (released documents appear here); no financial amounts shown

Main Success Scenario - EXPERT panel:

1. Actor sees: wallet available balance; per-milestone earned amounts; withdrawal history with bank confirmation timestamps; subscription tier + expiry; "Withdraw" button → UC27; "Upgrade Subscription" → UC14

Main Success Scenario - ADMIN panel:

1. Actor sees: full ledger read - `wallet_transactions`, `escrow_accounts`, `withdrawal_requests` (all users); no write actions

Postconditions: No state change - read-only financial dashboard.

Summary: <> and <> Relationship Index

Use this table as the authoritative reference for drawing the relationship arrows.

<> Relationships (Mandatory Sub-Behaviours)

Base UC	<> →	Rationale
UC01	Verify Subscription Gate	Called on every entry to elicitation engine
UC01	Run SDLC Void Detection	Always runs in Stage 1 LLM extraction
UC01	Run Automated Quality Gate	Always runs after Stage 5 synthesis
UC03	UC02 (conditional redirect)	Called when balance insufficient before subscription can activate
UC06d	UC20c	Surface C negotiation always requires expert response
UC11	UC30	Actor must browse marketplace before purchasing
UC11	UC02 (conditional redirect)	Called when wallet balance insufficient
UC14	UC02 (conditional redirect)	Called when balance insufficient
UC27	UC17 (conditional redirect)	Bank account must be linked before withdrawal

<> Relationships (Conditional/Optional Branches)

Extending UC	<> → Base UC	Extension Point / Condition
UC01a	UC01	Stage 4 unreachable + CEO has no TECH_TEAM
UC01b	UC01	<code>project.self_technical = true</code> detected in Stage 1

Extending UC	<> → Base UC	Extension Point / Condition
UC01t	UC01	TECH_TEAM drives Stage 4 independently (separate actor, conditional)
UC05t	UC05	Tier 2+ project (TECH_TEAM is responder instead of CEO)
UC06c	UC06b	bid.state = TECH_DISAPPROVED ; CEO wants to proceed
UC06d	UC06b	CEO opens price negotiation before approving selection
UC06r	UC06a	TECH_TEAM requests bid revision before final recommendation
UC10	UC10t	CEO budget approval follows TECH_TEAM technical approval
UC11	UC01a	TECH_DISCOVERY purchase as alternative to Technical Discovery milestone injection
UC19	UC20	Expert files spec clarifications before bid submission
UC20r	UC20	REVISION_REQUESTED state; only flagged component editable
UC20c	UC20	Price negotiation initiated by CEO after TECH_APPROVED
UC21	UC07	Expert accepts connection request (expert side of CEO's UC07)
UC26	UC24	SCOPE_EVOLUTION blocker flag triggers Add-On Phase Brief
UC-A1	UC-A2	Emergency pull-back triggered from monitoring view
UC30	UC11	CEO selects service to purchase from browse view
[Quality Gate fail path]	UC01	RETURNED_TO_CLIENT; re-entry at specific void
[Dispute filing]	UC09 or UC25	Filed from SUBMITTED or IN_REVISION milestone states